Formulários: fundamentos e campos HTML

Finalidade, estrutura, controles e validação nativa

Curso Técnico em Informática

Prof. Fabrício Rodrigues

O que vamos construir

Explicar para que serve um formulário e escolher os dados necessários.

Associar rótulos e distinguir id, name, type e value.

Construir escolhas únicas, múltiplas e campos com regras nativas.

Ponto de partida

Retome HTML, CSS e eventos vistos anteriormente. Se o primeiro formulário ficou pendente em 16/10, comece pelo campo de solicitante já fornecido nesta base.

Abra o projeto deste encontro no VS Code. Salve uma cópia própria antes de alterar.

Mapa da aula

Finalidade	Exemplo de uso
Cadastro ou inscrição	Registrar participação em uma atividade.
Busca ou filtro	Encontrar informações.
Contato ou suporte	Descrever uma solicitação.
No projeto da turma	Preparar um chamado fictício; sem envio real.

Fundamentos antes das tags

Um formulário permite informar dados e fazer escolhas para realizar uma tarefa. A finalidade orienta os campos: num chamado precisamos identificar o equipamento, classificar o problema e descrevê-lo.

Cadastro, login, pesquisa, contato e busca são finalidades. text, email e radio são tipos de controle; não são categorias de formulário. Nesta aula construiremos a estrutura HTML. CSS e JavaScript já fornecidos apenas mantêm o exemplo legível e a simulação local.

O papel de form, label e dos atributos

form reúne os controles da tarefa. label informa o que preencher. O for do label deve corresponder ao id do campo. Clique no rótulo para verificar a associação. Um placeholder é só um exemplo e não substitui o rótulo.

id identifica o elemento no documento. name identifica o dado na coleta. type escolhe o comportamento. value representa o valor do controle. Dois campos podem participar do mesmo grupo pelo name, mas seus ids devem ser únicos. Um código como PC-07 é texto, não uma quantidade.

Exemplo: O papel de form, label e dos atributos

```
<label for="contato">E-mail fictício (obrigatório)</label>
<input id="contato" name="contato" type="email" required>
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Uma lista de opções com select

select reúne opções conhecidas; cada option tem um valor de dados e um texto visível. A opção inicial com value vazio permite que required peça uma escolha.

No chamado, use hardware, software, rede e outro. Nesta aula, escolher outro ainda não abre um campo adicional. Esse comportamento será construído em 04/12.

Exemplo: Uma lista de opções com select

```
<label for="categoria">Categoria (obrigatório)</label>
<select id="categoria" name="categoria" required>
  <option value="">Escolha uma categoria</option>
  <option value="hardware">Hardware</option>
  <option value="software">Software</option>
  <option value="rede">Rede</option>
  <option value="outro">Outro</option>
</select>
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Escolha única com radio

Radios com o mesmo name formam um grupo de escolha única. Values diferentes distinguem as respostas. fieldset reúne o grupo e legend explica a pergunta.

Escolha prioridade normal ou alta. O required em um radio do grupo exige que uma opção daquele grupo seja escolhida. Não crie names diferentes para cada alternativa.

Exemplo: Escolha única com radio

```
<fieldset>
  <legend>Prioridade (obrigatória)</legend>
  <label><input type="radio" name="prioridade"
    value="normal" required> Normal</label>
  <label><input type="radio" name="prioridade"
    value="alta"> Alta</label>
</fieldset>
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Escolhas independentes com checkbox

Checkboxes permitem marcar nenhuma, uma ou várias opções independentes. Use sintomas opcionais: Lentidão e Mensagem de erro.

Os dois controles podem ter o mesmo name e values diferentes. Na coleta com JavaScript, usaremos getAll para recuperar todas as escolhas, em 06/11. required em um checkbox exige aquele checkbox específico; não significa “pelo menos um do conjunto”.

Exemplo: Escolhas independentes com checkbox

```
<fieldset>
  <legend>Sintomas (opcionais)</legend>
  <label><input type="checkbox" name="sintomas"
    value="Lentidão"> Lentidão</label>
  <label><input type="checkbox" name="sintomas"
    value="Mensagem de erro"> Mensagem de erro</label>
</fieldset>
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Texto longo e regras nativas

textarea recebe a descrição. A instrução informa de 20 a 500 caracteres, e aria-describedby associa a ajuda ao campo. required, minlength e maxlength são regras que o navegador pode conferir.

type=email verifica um formato, sem comprovar a existência da caixa de e-mail. As mensagens variam por navegador. A versão desta aula testa as regras HTML; comparação entre e-mails e texto formado só por espaços ficam para 13/11.

Exemplo: Texto longo e regras nativas

```
<label for="descricao">Descrição (obrigatória)</label>
<p id="ajuda-descricao">Use de 20 a 500 caracteres.</p>
<textarea id="descricao" name="descricao" rows="5"
  required minlength="20" maxlength="500"
  aria-describedby="ajuda-descricao"></textarea>
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Atividade no VS Code

Baixar o projeto desta aula, extrair os três arquivos e abrir a pasta no VS Code. Executar index.html com Live Server.

Usar o campo de solicitante como modelo. Acrescentar equipamento (texto, 2 a 20 caracteres), contato (email), categoria (select), prioridade (radio), descrição (20 a 500 caracteres) e os dois sintomas opcionais (checkbox).

Atividade: conferência e explicação

Associar rótulos, usar names coerentes e deixar visível o que é obrigatório. Manter o script de simulação fornecido.

Salvar, testar os casos e explicar a diferença entre id e name usando um campo do seu projeto.

Conferência dos testes

Caso	Resultado esperado
Campos obrigatórios vazios	O navegador impede a simulação.
Contato sem formato de e-mail	O campo de contato fica inválido.
Descrição digitada com menos de 20 caracteres	O navegador pede completar o texto.
Clique no rótulo do equipamento	O foco vai para o campo correspondente.
Marcar duas prioridades em sequência	Só a última permanece marcada.

Conferência dos testes (continuação)

Caso	Resultado esperado
Marcar os dois sintomas	As duas opções permanecem marcadas.
Preenchimento válido	Mensagem local; nenhuma navegação de envio.

Erros que merecem atenção

name não substitui id na associação com label.

Um select com opção inicial já válida não testa a escolha obrigatória.

Não acrescente novalidate: queremos observar a validação HTML.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

Qual é o papel de name? Qual teste mostrou que seu formulário estava funcionando?

Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

Um formulário legível em qualquer tela

CSS, responsividade, instruções e navegação pelo teclado

Curso Técnico em Informática

Prof. Fabrício Rodrigues

O que vamos construir

Organizar os campos em uma e duas colunas sem rolagem horizontal.

Manter rótulos, instruções e foco legíveis.

Testar teclado e ampliação sem depender somente da aparência.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Abra o projeto deste encontro no VS Code. Salve uma cópia própria antes de alterar.

Mapa da aula

Camada	Responsabilidade
HTML	Preserva rótulos, grupos e regras.
CSS	Organiza largura, espaçamento e foco.
Teclado	Permite percorrer e acionar os controles.
Conferência	Telas estreitas, ampliação e sequência de foco.

Layout começa pela tela estreita

Abra a base com o HTML completo da aula anterior. A regra global box-sizing inclui borda e preenchimento na largura calculada. Evite campos com largura fixa maior que a tela.

Uma grade com gap organiza os campos em uma coluna. O código fornecido já mantém inputs com largura de 100% e min-width:0. O objetivo da aula é completar o layout sem alterar os names e as regras HTML.

Exemplo: Layout começa pela tela estreita

```
* { box-sizing: border-box; }
.campos {
  display: grid;
  gap: 1.2rem;
}
.campo { min-width: 0; }
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Duas colunas quando há espaço

A media query entra a partir de 48rem. Até esse limite, a grade continua em uma coluna. O campo de descrição ocupa a linha inteira por meio da classe largo.

O ponto de mudança pertence a este projeto. Ele não define um tipo fixo de aparelho: observe quando os campos deixam de caber com conforto.

Exemplo: Duas colunas quando há espaço

```
@media (min-width: 48rem) {
  .campos { grid-template-columns: 1fr 1fr; }
  .largo { grid-column: 1 / -1; }
}
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Foco visível e ordem de leitura

Tab percorre os controles e Shift+Tab retorna. A ordem do HTML deve acompanhar a ordem de leitura; não use tabindex positivo para reorganizar o formulário.

focus-visible mostra uma indicação clara para navegação. Não remova o contorno sem oferecer indicação equivalente. Rótulos e instruções continuam visíveis durante o preenchimento.

Exemplo: Foco visível e ordem de leitura

```
:focus-visible {
  outline: 3px solid #6a8f32;
  outline-offset: 4px;
}
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Teste da experiência

Teste aproximadamente 360px, 768px e 1280px, além da ampliação do navegador em 200%. Compare largura, tamanho dos textos, quebra das instruções e acesso aos botões.

No modo responsivo do navegador, altere a largura e observe o resultado. Medir apenas a quantidade de colunas não comprova que o formulário ficou utilizável.

Atividade no VS Code

Abrir a base desta aula ou continuar o HTML concluído no encontro anterior.

Completar a media query para duas colunas e fazer a descrição ocupar a largura da grade.

Atividade: conferência e explicação

Acrescentar foco visível e revisar espaçamentos dos grupos e das instruções.

Pedir a um colega que percorra o formulário pelo teclado e repetir os testes nas larguras indicadas.

Conferência dos testes

Caso	Resultado esperado
Tela de aproximadamente 360px	Uma coluna; controles sem corte lateral.
Tela larga a partir de 48rem	Duas colunas; descrição ocupa a linha.
Tab e Shift+Tab	Foco visível, sequência compreensível.
Ampliação de 200%	Conteúdo legível e controles alcançáveis.
Texto longo na instrução	Quebra de linha sem sobrepor outro campo.

Conferência dos testes (continuação)

Caso	Resultado esperado
Preenchimento inválido	Regras HTML da aula anterior continuam ativas.

Erros que merecem atenção

Trocar a aparência de um campo não deve retirar seu rótulo.

Uma largura fixa de 900px pode quebrar a versão estreita.

Não usar cor como única indicação de erro ou obrigatoriedade.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

Qual decisão ajudou a tela estreita? Como você conferiu o foco?

Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

Os dados aparecem na página

Evento submit, FormData e resumo local com JavaScript

Curso Técnico em Informática

Prof. Fabrício Rodrigues

O que vamos construir

Escutar submit no formulário e impedir a navegação na simulação.

Coletar campos por name, preservando múltiplos checkboxes.

Montar um resumo com elementos do DOM e textContent.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Abra o projeto deste encontro no VS Code. Salve uma cópia própria antes de alterar.

Mapa da aula

Passo	O que acontece
Preencher	A pessoa informa dados fictícios.
Validar	O navegador aplica as regras HTML.
submit	O evento chega ao formulário válido.
JavaScript	Impede a navegação e apresenta o resumo local.

O evento pertence ao formulário

Usar submit permite tratar o botão e a submissão pelo teclado. A validação HTML interativa acontece antes do submit: campos inválidos podem impedir que esse evento aconteça.

preventDefault interrompe a ação padrão nesta demonstração. Ele não valida, não envia para um servidor e não grava os dados. A base já oferece um bloqueio de navegação; substitua a mensagem pelo processamento do resumo.

Exemplo: O evento pertence ao formulário

```
form.addEventListener('submit', (event) => {
  event.preventDefault();
  const dados = new FormData(form);
  mostrarResumo(dados);
});
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Coleta por name

new FormData(form) monta os pares de dados dos controles participantes. get recupera um valor; getAll recupera todos os valores com o mesmo name. Checkboxes desmarcados não entram na coleta.

Um campo com id e sem name continua aparecendo na tela, mas não entra como par nome/valor na coleta. Os sintomas selecionados precisam aparecer juntos no resumo.

Exemplo: Coleta por name

```
const dados = new FormData(form);
const contato = dados.get('contato');
const sintomas = dados.getAll('sintomas');
// getAll preserva várias opções com o mesmo name.
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Resumo como conteúdo, sem interpretar HTML

Crie dt e dd para cada item da lista de descrição. textContent apresenta o valor como texto. Não use innerHTML para inserir o que a pessoa digitou.

Limpe o resumo anterior antes de montar o novo. Depois torne a seção visível e leve o foco ao título com tabindex=-1. O resumo corresponde à última simulação, não a um registro persistente.

Exemplo: Resumo como conteúdo, sem interpretar HTML

```
function adicionarLinha(titulo, valor) {
  const dt = document.createElement('dt');
  const dd = document.createElement('dd');
  dt.textContent = titulo;
  dd.textContent = String(valor);
  resumo.append(dt, dd);
}
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Estado do formulário depois da tentativa

Uma falha de validação deve manter os valores para correção. Um novo envio deve substituir o resumo anterior, sem duplicar linhas.

O botão Novo preenchimento é type=button. Ele pode chamar reset, ocultar o resumo e devolver o foco ao primeiro campo. Avise que o projeto perde os dados quando a página é recarregada.

Atividade no VS Code

Abrir a base da aula com HTML e CSS concluídos.

Substituir o tratamento de submit pela coleta com FormData e pela chamada que monta o resumo.

Atividade: conferência e explicação

Mostrar solicitante, equipamento, contato, categoria, prioridade, descrição e todos os sintomas selecionados.

Acrescentar a limpeza do resumo em uma nova simulação e o botão de novo preenchimento.

Conferência dos testes

Caso	Resultado esperado
Campo obrigatório vazio	O resumo não aparece.
Preenchimento válido	Resumo local com os valores informados.
Dois sintomas marcados	Ambos aparecem no resumo.
Nenhum sintoma marcado	Mensagem de nenhuma opção selecionada.
Texto com exemplo na descrição	As marcas aparecem como texto, sem formatação HTML.

Conferência dos testes (continuação)

Caso	Resultado esperado
Duas simulações válidas	O resumo mais recente substitui o anterior.
Novo preenchimento	Campos reiniciados e foco no primeiro campo.

Erros que merecem atenção

Escutar somente click do botão deixa de representar a tarefa do formulário.

get recupera um único valor; sintomas exigem getAll.

Resumo na tela não significa que o chamado foi enviado ou salvo.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

Como o name chega ao FormData? Por que usar textContent no resumo?

Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

Mensagens que ajudam a corrigir

Regras complementares, validade personalizada e casos de teste

Curso Técnico em Informática

O que vamos construir

Comparar o contato com sua confirmação sem perder as regras nativas.

Criar e limpar mensagens com setCustomValidity.

Registrar casos válidos, inválidos e limites da descrição.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Abra o projeto deste encontro no VS Code. Salve uma cópia própria antes de alterar.

Mapa da aula

Regra	Camada do protótipo
Obrigatoriedade e formato	Atributos HTML.
Confirmação de contato	JavaScript compara dois campos.
Descrição útil	JavaScript confere o texto sem espaços nas pontas.
Aplicação real	Servidor precisaria conferir as regras novamente.

Comparação de dois campos

A base acrescenta o campo de confirmação com type=email e required. Isso confere cada formato, mas não prova que os dois valores são iguais.

A função compara os valores e define uma mensagem quando diferem. String vazia limpa a validade personalizada. Escute input nos dois campos, pois qualquer um pode mudar depois.

Exemplo: Comparação de dois campos

```
function conferirContato() {
  const diferente = confirmar.value !== contato.value;
  confirmar.setCustomValidity(
    confirmar.value && diferente ? 'Repita o mesmo e-mail.' : ''
  );
}
contato.addEventListener('input', conferirContato);
confirmar.addEventListener('input', conferirContato);
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Descrição e limites do teste

minlength e maxlength tratam o comprimento do valor. Nesta aula acrescentamos a regra de pelo menos 20 caracteres depois de trim, para não aceitar uma descrição formada só por espaços.

A função limpa a mensagem quando o texto passa. Teste digitando no campo; restrições de comprimento nativas têm particularidades quando um script atribui o valor diretamente. Não altere o valor do aluno em silêncio.

Exemplo: Descrição e limites do teste

```
function conferirDescricao() {
  const curta = descricao.value.length > 0
    && descricao.value.trim().length < 20;
  descricao.setCustomValidity(
    curta ? 'Descreva o problema com pelo menos 20 caracteres.' : ''
  );
}
descricao.addEventListener('input', conferirDescricao);
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Nova tentativa sem mensagens antigas

Antes de montar o resumo, atualize as verificações e consulte reportValidity. Os listeners de input são essenciais para limpar um erro anterior que bloquearia a próxima submissão.

Ao reiniciar, faça reset e limpe a validade personalizada. Mostrar um aviso não deve apagar os campos nem transferir a culpa para a pessoa. A mensagem precisa indicar como corrigir.

Exemplo: Nova tentativa sem mensagens antigas

```
form.addEventListener('submit', (event) => {
  event.preventDefault();
  conferirContato();
  conferirDescricao();
  if (!form.reportValidity()) return;
  mostrarResumo(new FormData(form));
});
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Atividade no VS Code

Abrir a base e localizar os campos contato, confirmar e descricao.

Implementar a comparação e registrar input nos dois e-mails.

Atividade: conferência e explicação

Acrescentar a regra da descrição com trim e limpar as mensagens quando o valor for corrigido.

Repetir a tentativa após cada correção e registrar os limites e o reinício.

Conferência dos testes

Caso	Resultado esperado
E-mails diferentes	Mensagem de repetir o mesmo contato.
Corrigir o segundo e-mail	A mensagem antiga é limpa.
Mudar o primeiro depois da confirmação	A divergência volta a ser identificada.
Descrição formada por espaços	A regra complementar bloqueia a simulação.
Descrição digitada com 19 caracteres úteis	A simulação fica bloqueada.

Conferência dos testes (continuação)

Caso	Resultado esperado
Descrição digitada com 20 caracteres úteis	A regra de tamanho permite continuar.
Novo preenchimento após erro	Não permanece mensagem personalizada antiga.

Erros que merecem atenção

Mensagem personalizada sem limpeza mantém o campo inválido.

Comparar apenas no click não cobre todas as formas de tentativa.

Validação no navegador não substitui a conferência em um servidor real.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

Quando sua mensagem é limpa? Que teste demonstrou a correção do erro anterior?

Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

O caminho dos dados

GET, POST, HTTPS e contrato do projeto integrador

Curso Técnico em Informática

Prof. Fabrício Rodrigues

O que vamos construir

Distinguir uma simulação local de uma requisição a um serviço.

Comparar GET e POST e explicar o papel do HTTPS.

Documentar campos e regras de um serviço futuro sem inventar um backend.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Abra o projeto deste encontro no VS Code. Salve uma cópia própria antes de alterar.

Mapa da aula

Situação	Representação
Busca de dados com GET	Parâmetros podem aparecer na URL.
Envio com POST	Dados de formulário podem seguir no corpo.
HTTPS	Protege o tráfego em trânsito.
Projeto no GitHub Pages	Site estático; não implementa recebimento de chamados.

Os limites do protótipo

Até aqui o JavaScript trabalha somente na página. Mostrar um resumo não faz uma requisição de envio. O GitHub Pages publica os arquivos estáticos do curso; não executa um serviço próprio para receber chamados.

Uma aplicação real precisaria de uma rota de servidor e de regras de recebimento. Nesta aula documentaremos esse contrato e compararemos representações, sem enviar dados a um endereço externo.

GET e os parâmetros da URL

GET costuma ser usado para recuperar informações. Em um formulário com method=get, os pares de dados podem compor a query da URL. Eles podem aparecer no histórico e ser compartilhados junto com o endereço.

O exemplo gera uma string com URLSearchParams e a mostra como texto. exemplo.test é um domínio de exemplo; o projeto não navega para ele. Observe como nomes, espaços e caracteres são representados.

Exemplo: GET e os parâmetros da URL

```
const dados = new FormData(form);
const parametros = new URLSearchParams(dados);
const exemplo = 'https://exemplo.test/chamados?'
  + parametros.toString();
// Apenas exibir o texto. Não navegar para esse endereço.
```

Observe o papel de cada linha e compare com o arquivo do projeto.

POST, HTTPS e conferência no servidor

Em um formulário HTML com method=post, os dados podem seguir no corpo da requisição. POST sozinho não criptografa dados. HTTPS protege o tráfego em trânsito. Não coloque informações sensíveis em uma URL por achar que o método resolveu toda a proteção.

O servidor deve conferir formato, tamanho, escolhas permitidas e demais regras. O usuário pode alterar a página ou enviar outra requisição. A validação HTML e o JavaScript servem à experiência local e precisam de conferência no serviço real.

Contrato do integrador

Documente no HTML um bloco de contrato: finalidade, método escolhido, rota proposta, campos e regras a aplicar no servidor. Identifique claramente que a rota não está implementada.

A base exibe exemplos de URL GET e de um corpo para POST depois da simulação. São representações de estudo. Complete o contrato e confronte seus nomes de campos com o que FormData realmente coleta.

Exemplo: Contrato do integrador

```
<!-- Exemplo de contrato para uma aplicação futura.
Método: POST. Rota proposta: /chamados.
O servidor deve validar os campos novamente.
Este projeto estático não implementa essa rota.
-->
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Atividade no VS Code

Abrir a base da aula e observar o resumo e as representações de estudo.

Comparar onde os valores apareceriam em GET e POST e explicar por que não houve envio real.

Atividade: conferência e explicação

Completar o contrato no HTML: finalidade, POST, rota proposta /chamados, campos, regras e indicação de serviço ainda não implementado.

Conferir se os names coletados correspondem ao contrato e registrar o escopo do integrador.

Conferência dos testes

Caso	Resultado esperado
Simulação com dados fictícios	Resumo e representações locais, sem envio.
Espaços no nome	URL de exemplo com parâmetros codificados.
Dois sintomas selecionados	Representação de POST preserva a lista.
Contrato sem indicar implementação futura	Revisar para não prometer serviço existente.
POST sem HTTPS no argumento	Explicar que POST não oferece criptografia.

Conferência dos testes (continuação)

Caso	Resultado esperado
Campo sem name	Identificar a ausência na coleta e corrigir o contrato.

Erros que merecem atenção

O corpo de POST não é uma garantia de criptografia.

Não afirmar que o Pages recebeu, validou ou armazenou o chamado.

Um contrato é uma proposta; não é prova de que o endpoint existe.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

Qual parte do seu projeto roda no navegador? O que ainda faltaria num serviço real?

Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

Um formulário que reage ao contexto

Campos condicionais, estado e organização das funções

Curso Técnico em Informática

O que vamos construir

Mostrar um campo adicional somente quando sua opção for escolhida.

Coordenar hidden, disabled e required.

Repetir a coleta e o reinício sem valores de uma opção antiga.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Abra o projeto deste encontro no VS Code. Salve uma cópia própria antes de alterar.

Mapa da aula

Categoria	Estado do campo adicional
Outra opção não escolhida	Bloco oculto, campo desabilitado e não obrigatório.
Outro escolhido	Bloco visível, campo habilitado e obrigatório.
Retorno a uma categoria comum	Limpar o valor antigo e desativar o campo.
Coleta	Somente os campos participantes entram no FormData.

A interface responde a uma escolha

O HTML da base já contém o bloco de outra categoria e um input inicialmente disabled. A função de estado deve ler o select e atualizar o conjunto de propriedades.

Registre change no select e chame a função ao iniciar a página. Organize a regra em uma função, para também aplicá-la depois de reset. Não espalhe a mesma decisão em vários handlers.

Exemplo: A interface responde a uma escolha

```
function atualizarOutro() {
  const ativo = categoria.value === 'outro';
  blocoOutro.hidden = !ativo;
  outro.disabled = !ativo;
  outro.required = ativo;
  if (!ativo) outro.value = '';
}
categoria.addEventListener('change', atualizarOutro);
atualizarOutro();
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Esconder não desativa o controle

hidden altera a apresentação do bloco. Um campo obrigatório somente escondido ainda pode interferir na validação. disabled retira o controle da validação e da coleta normal de dados.

required deve ficar ativo apenas quando o campo é necessário. Ao desativar a opção, o projeto apaga o valor antigo para não sugerir um dado que deixou de pertencer à escolha atual.

Exemplo: Esconder não desativa o controle

```
// hidden controla a apresentação do bloco.
// disabled retira o campo da validação e do FormData.
// required pede um valor quando o campo está ativo.
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Conferência do fluxo inteiro

Teste entrar em Outro, tentar sem preencher, informar um detalhe e gerar o resumo. Depois retorne a Hardware e confira que o detalhe não aparece na coleta.

Use Novo preenchimento para conferir reset e reaplicação do estado. Revise a comparação dos e-mails e as mensagens: acrescentar uma condição não deve quebrar as regras anteriores.

Atividade no VS Code

Abrir a base e localizar o select, o bloco oculto e o campo adicional.

Criar a função de estado, escutar change e aplicá-la na inicialização.

Atividade: conferência e explicação

Manter o campo adicional obrigatório apenas em Outro e incluir seu valor no resumo somente quando coletado.

Testar todas as transições e executar novamente os casos de validação das aulas anteriores.

Conferência dos testes

Caso	Resultado esperado
Abertura com categoria vazia	Campo adicional oculto e desabilitado.
Escolher Outro sem detalhar	Campo visível e obrigatório.
Outro com detalhe e dados válidos	Resumo inclui a outra categoria.
Trocar de Outro para Hardware	Bloco oculto e valor antigo limpo.
Gerar resumo após sair de Outro	Não inclui outra_categoria.

Conferência dos testes (continuação)

Caso	Resultado esperado
Novo preenchimento	Estado inicial reaplicado.

Erros que merecem atenção

hidden sozinho não expressa a participação do campo.

Não deixar required ativo num campo desabilitado por lógica incompleta.

reset não substitui a reaplicação explícita da regra de estado.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

Que diferença existe entre esconder um campo e desabilitá-lo?

Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

Avaliação prática: inscrição em oficina

Aplicação individual dos conhecimentos de formulários

Curso Técnico em Informática

Prof. Fabrício Rodrigues

O que vamos construir

Construir um formulário para um contexto diferente do chamado.

Demonstrar estrutura, responsividade, coleta e validação.

Registrar testes e explicar uma decisão de implementação.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Abra o projeto deste encontro no VS Code. Salve uma cópia própria antes de alterar.

Mapa da aula

Critério	Valor proposto
HTML e controles adequados	2 pontos.
CSS, rótulos e teclado	2 pontos.
Coleta e resumo local	2 pontos.
Validação e campo condicional	2 pontos.
Testes e explicação individual	2 pontos.

Situação da avaliação

Uma oficina de informática precisa de um protótipo de inscrição. Use somente dados fictícios e mantenha o envio simulado. O professor explicará as regras de consulta e os critérios antes do início.

O participante deve escolher uma oficina (Redes, Hardware, Web ou Outra oficina) e um turno (Manhã ou Tarde). Materiais que levará são opcionais: Caderno e Notebook.

Especificação dos campos

Participante fictício: texto de 2 a 60 caracteres. Contato e confirmação: formato de e-mail, obrigatórios e iguais. Oficina: select obrigatório com opção inicial vazia. Turno: uma opção obrigatória em radio.

Objetivo da participação: de 20 a 500 caracteres úteis. Ao escolher Outra oficina, mostrar Tema sugerido, habilitá-lo e torná-lo obrigatório. Ao sair dessa escolha, limpar e desabilitar esse campo.

Comportamento e entrega

Mostrar um resumo local com os dados da inscrição, preservando os dois materiais opcionais se marcados. Tratar valores como texto. Não enviar nem gravar em servidor.

O formulário deve funcionar em tela estreita e pelo teclado. Manter valores após uma tentativa inválida. O reinício precisa limpar as mensagens e o estado condicional. Entregar os três arquivos em ZIP e o registro TXT.

Atividade no VS Code

Ler os requisitos e os critérios com o professor. Baixar a base da avaliação.

Construir individualmente o HTML, completar CSS e JavaScript conforme a especificação.

Atividade: conferência e explicação

Realizar os testes da tabela e registrar os resultados reais, inclusive o que ainda falhou.

Entregar ZIP e TXT e explicar um campo, uma regra e um teste ao professor.

Conferência dos testes

Caso	Resultado esperado
Obrigatórios vazios	Bloqueio da tentativa.
E-mails diferentes e depois corrigidos	Erro comunicado e mensagem limpa ao corrigir.
Outra oficina sem tema	Tema obrigatório.
Trocar de Outra oficina para Web	Tema deixa a validação e a coleta.
Caderno e Notebook marcados	Resumo preserva os dois materiais.

Conferência dos testes (continuação)

Caso	Resultado esperado
Texto com sinais de tag no objetivo	Aparece como texto literal.
Tela estreita e teclado	Campos alcançáveis e foco visível.
Novo preenchimento	Regras e estado voltam à condição inicial.

Erros que merecem atenção

A entrega visual não substitui a explicação do código.

Não importar o resumo completo do projeto de chamado sem adaptar os names.

O registro de teste precisa distinguir previsão e resultado observado.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

Qual requisito você demonstrou com maior autonomia? Qual precisa corrigir?

Guarde a versão desta aula: ela será o ponto de partida para a continuidade.

Corrigir, recuperar e concluir

Devolutiva, depuração e nova evidência de aprendizagem

Curso Técnico em Informática

Prof. Fabrício Rodrigues

O que vamos construir

Interpretar a devolutiva com base em critérios e testes.

Localizar e corrigir defeitos de rótulo, coleta e estado.

Demonstrar novamente os critérios pendentes.

Ponto de partida

Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Abra o projeto deste encontro no VS Code. Salve uma cópia própria antes de alterar.

Mapa da aula

Etapa	Evidência
Devolutiva	Critério, resultado observado e ponto a corrigir.
Correção	Hipótese, alteração e novo teste.
Recuperação	Tarefa equivalente do critério pendente.
Fechamento	Explicação individual e versão corrigida.

Devolutiva sem presumir os resultados

O professor devolve a avaliação real com os critérios e as evidências observadas. Não existe uma nota antecipada neste material. Escolha o que precisa retomar a partir dessa devolutiva.

O projeto de recuperação contém defeitos intencionais para investigar. Quem já demonstrou os critérios pode explicar os defeitos e ampliar os testes, conforme orientação do professor.

Investigação de rótulos e coleta

Clique no rótulo de contato e observe o foco. Compare for com o id. Depois confira se o campo possui name e se a coleta usa o formulário correto.

Faça uma mudança por vez e repita o mesmo teste. Um formulário vazio de dados no resumo pode indicar um problema de coleta, mesmo que os campos tenham sido preenchidos.

Exemplo: Investigação de rótulos e coleta

```
const dados = new FormData(form);
const contato = dados.get('contato');
const sintomas = dados.getAll('sintomas');
// getAll preserva várias opções com o mesmo name.
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Estado e nova tentativa

Alterne a opção Outra oficina e confira visibilidade, obrigatoriedade e participação do tema sugerido. Compare as propriedades com a tabela de estados estudada.

Documente o problema, a hipótese, a alteração feita e o resultado depois da correção. Use uma tarefa equivalente para demonstrar os critérios ainda pendentes; siga as regras de recuperação da turma.

Exemplo: Estado e nova tentativa

```
// hidden controla a apresentação do bloco.
// disabled retira o campo da validação e do FormData.
// required pede um valor quando o campo está ativo.
```

Observe o papel de cada linha e compare com o arquivo do projeto.

Atividade no VS Code

Ler a devolutiva e escolher os critérios a retomar com o professor.

Abrir a base de recuperação e investigar os defeitos de rótulo, name, coleta e campo condicional.

Atividade: conferência e explicação

Corrigir uma hipótese por vez e repetir os testes.

Entregar a versão corrigida com o registro do progresso e explicar uma correção sem ler a solução.

Conferência dos testes

Caso	Resultado esperado após a correção
Clicar no rótulo de contato	O foco chega ao contato.
Preencher contato e gerar resumo	O valor informado aparece no resumo.
Escolher Outra oficina	Tema fica habilitado e obrigatório.
Sair de Outra oficina	Tema fica oculto e fora da coleta.
Corrigir um valor inválido	A próxima tentativa pode ocorrer.

Conferência dos testes (continuação)

Caso	Resultado esperado após a correção
Repetir o caso da avaliação	A evidência do critério pendente fica registrada.

Erros que merecem atenção

Trocar vários trechos ao mesmo tempo dificulta identificar a causa.

Uma mensagem de sucesso sozinha não comprova o conteúdo do resumo.

Recuperação deve produzir nova evidência do critério, conforme as regras da turma.

Registro e entrega

Salve index.html, style.css e script.js. Compacte a pasta em ZIP.

Abra o registro de testes da aula, informe nome e turma e baixe o TXT.

Entregue ZIP e TXT ao professor na aula. Os downloads não são enviados automaticamente.

Uma explicação para fechar

Qual era a causa do erro, como você a corrigiu e qual teste confirmou a mudança?

Guarde a versão desta aula: ela será o ponto de partida para a continuidade.